

torch.autograd.functional.jvp#

<https://pytorch.org/docs/stable/generated/torch.autograd.functional.jvp.html>

Description:

Compute the dot product between the Jacobian of the given function at the point given by the inputs and a vector v .

func (function) – a Python function that takes Tensor inputs and returns a tuple of Tensors or a Tensor.

inputs (tuple of Tensors or Tensor) – inputs to the function **func**.

v (tuple of Tensors or Tensor) – The vector for which the Jacobian vector product is computed. Must be the same size as the input of **func**. This argument is optional when the input to **func** contains a single element and (if it is not provided) will be set as a Tensor containing a single 1.

create_graph (bool, optional) – If True, both the output and result will be computed in a differentiable way. Note that when **strict** is False, the result can not require gradients or be disconnected from the inputs. Defaults to False.

Function Signature

```
torch.autograd.functional.jvp(func, inputs, v=None,  
create_graph=False, strict=False)[source]#
```

Parameters

Returns

tuple with:

Return type

Returns:

output (tuple)

Code Examples

```
>>> def exp_reducer(x):
...     return x.exp().sum(dim=1)
>>> inputs = torch.rand(4, 4)
>>> v = torch.ones(4, 4)
>>> jvp(exp_reducer, inputs, v)
(tensor([6.3090, 4.6742, 7.9114, 8.2106]),
 tensor([6.3090, 4.6742, 7.9114, 8.2106]))
```

```
>>> jvp(exp_reducer, inputs, v, create_graph=True)
(tensor([6.3090, 4.6742, 7.9114, 8.2106], grad_fn=
<SumBackward1>),
 tensor([6.3090, 4.6742, 7.9114, 8.2106], grad_fn=
<SqueezeBackward1>))
```

```
>>> def adder(x, y):
...     return 2 * x + 3 * y
>>> inputs = (torch.rand(2), torch.rand(2))
>>> v = (torch.ones(2), torch.ones(2))
>>> jvp(adder, inputs, v)
(tensor([2.2399, 2.5005]),
 tensor([5., 5.]))
```

Notes & Warnings

NOTE

Note `autograd.functional.jvp` computes the jvp by using the backward of the backward (sometimes called the double backwards trick). This is not the most performant way of computing the jvp. Please consider using `torch.func.jvp()` or the low-level forward-mode AD API instead.

Detailed Documentation

Documentation

Compute the dot product between the Jacobian of the given function at the point given by the inputs and a vector `v`.

`func` (function) – a Python function that takes Tensor inputs and returns a tuple of Tensors or a Tensor.

`inputs` (tuple of Tensors or Tensor) – inputs to the function `func`.

`v` (tuple of Tensors or Tensor) – The vector for which the Jacobian vector product is computed. Must be the same size as the input of `func`. This argument is optional when the input to `func` contains a single element and (if it is not provided) will be set as a Tensor containing a single 1.

`create_graph` (bool, optional) – If True, both the output and result will be computed in a differentiable way. Note that when `strict` is False, the result can not require gradients or be disconnected from the inputs. Defaults to False.

strict (bool, optional) – If True, an error will be raised when we detect that there exists an input such that all the outputs are independent of it. If False, we return a Tensor of zeros as the jvp for said inputs, which is the expected mathematical value. Defaults to False.

func_output (tuple of Tensors or Tensor): output of func(inputs)

jvp (tuple of Tensors or Tensor): result of the dot product with the same shape as the output.

autograd.functional.jvp computes the jvp by using the backward of the backward (sometimes called the double backwards trick). This is not the most performant way of computing the jvp. Please consider using `torch.func.jvp()` or the low-level forward-mode AD API instead.